Aufgabe 2: Simultane Labyrinthe

Teilnahme-ID: 76823

Bearbeiter/-in dieser Aufgabe:
Philipp Strieder

28. April 2025

Inhaltsverzeichnis

1 Lösungsidee **1**
1.1 Lösung 1: BFS . . . 1
1.2 Lösung 2: A* . . . 1
1.3 Lösung 3: Heuristik . . . 2

2 Laufzeit- und Speicheranalyse **2**

3 Umsetzung **2**

4 Erweiterung **3**

5 Beispiele **3**

6 Quellcode **3**

1 Lösungsidee

1.1 Lösung 1: BFS

Ansatz 1 verfolgt eine Breitensuche (Breadth-first-search). Dafür erstellen wir einen gerichteten Graphen, in dem die Knoten die Positionenpaare sind und eine Kante von einem Knoten auf einen anderen genau dann existiert, wenn man mit einem Z̈ugv̈on der Position des ersten Knotens auf die Position des zweiten kommen kann. Über diesen Graphen lassen wir dann eine BFS laufen. Wir beginnen am Startknoten (Das ist die Position $((0,0),(0,0))$). In jedem Schritt speichern wir dann alle Knoten, die wir von den bisher erreichten erreichen können, solange bis wir entweder keine neuen Knoten mehr finden oder der Zielknoten (Die Position $((m,n),(m,n))$) erreicht haben. Zusätzlich tracken wir zu jedem neuen Knoten den Pfad zu ihm und geben dann am Ende den Pfad zum Zielknoten aus. Diese Suche garantiert die kürzeste Lösung, ist aber sehr speicherlastig, da blind alle Kombinationen gespeichert werden

1.2 Lösung 2: A*

Die zweite Lösung verwendet eine A*-Suche. Wir wandeln das Labyrinth erneut wie oben in einen Graphen um. Für jeden Zustand wird eine Kostenfunktion $f = g + h$ berechnet, wobei g die bisher zurückgelegten Schritte und h die geschätzten verbleibenden Kosten bis zum Ziel darstellt. Als Heuristik wird der größere der beiden tatsächlichen Distanzen der Spielfiguren zum Ziel gewählt – diese Distanzen wurden zuvor anhand einer rückwärtsgerichteten BFS vom Ziel aus ermittelt. Diese Heuristik ist immer maximal genau so groß, wie die tatsächlichen Kosten, was Bedingung für A* ist. Der Algorithmus erkundet immer die vielversprechendsten Zustände (bei denen die Heuristik die geringsten Werte liefert) zuerst und spart im Vergleich zur reinen BFS erheblich an Rechenzeit, da viele Wege ignoriert werden können, da sie im Vergleich zu einer ersten gefundenen Lösung definitiv zu langsam sind.

1.3 Lösung 3: Heuristik

Die dritte Lösung verfolgt einen rein heuristischen Ansatz. Statt global zu planen, trifft sie in jedem Schritt eine lokale Entscheidung: Von allen möglichen gleichzeitigen Bewegungen wird diejenige ausgewählt, bei der die Summe der verbleibenden Distanzen beider Figuren am stärksten sinkt – basierend auf denselben, zuvor per BFS berechneten Distanzen. Dieser Greedy-Ansatz ist sehr effizient und verbraucht kaum Speicher, kann aber suboptimale oder sogar fehlerhafte Wege produzieren, da keine Garantie für eine vollständige Lösung besteht. Insbesondere in komplexeren Labyrinthen kann es vorkommen, dass der Algorithmus keinen Ausweg findet, wenn er zum Beispiel zwischen Feldern gefangen ist, die er alle schon besucht hat.

2 Laufzeit- und Speicheranalyse

Es ist $N = nm$ die Anzahl der Felder eines Labyrinths.
Der BFS erkundet im worst-case alle $O(N^2)$ 4-Tupel einmal. Da das Innere der Schleife in $O(1)$ laufen sollte, ist die Gesamtlaufzeit $O(N^2)$. Da jedes Tupel auf konstantem Speicher gespeichert werden kann ist der Speicherverbrauch auch $O(N^2)$.
Der A*-Algorithmus muss im worst-case auch $O(N^2)$ Zustände durchgehen. Allerdings hat das Innere der Schleife nicht mehr konstante Laufzeit, sondern $O(\log(N^2)) = O(\log(N))$. Damit ergibt sich die Gesamtlaufzeit $O(N^2 \log(N))$. Dieser worst-case wird allerdings selten erreicht, weshalb in der Praxis A* oft schneller ist. Der Speicherverbrauch ist im worst-case wieder $O(N^2)$, in der Praxis aber oft geringer.
Der greedy-Algorithmus trifft im worst-case $O(N)$ Entscheidungen, welche Richtung er wählen soll. Da die Distanzen zum Ziel in Laufzeit $O(N)$ schon vorberechnet wurden, läuft das Innere der Schleife in $O(1)$ und die Gesamtlaufzeit beträgt $O(N)$

3 Umsetzung

Die Implementierung der drei Ansätze basiert jeweils auf einer vorberechneten Darstellung der erlaubten Bewegungen im Labyrinth.

Für die BFS-Variante wird zunächst eine Lookup-Tabelle `reach` erzeugt, die für jedes Labyrinth von jeder Position aus das Ergebnis alle möglichen Bewegungen (rechts, unten, links, oben) speichert. Der BFS-Algorithmus speichert Zustände als Paare der beiden Figurenpositionen. Da Python bei Tupeln einen overhead erzeugt, werden zur Entlastung des Speichers Zustände mithilfe einer Komprimierungsfunktion `compress_state` in Ganzzahlen umgewandelt. Während der Suche wird in jeder Iteration die aktuelle Queue abgearbeitet, neue Zustände erzeugt und, falls noch nicht gesehen, in die Queue eingefügt.

Beim A*-Algorithmus wird zusätzlich zu `reach` (die möglichen Bewegungen) eine Distanzfunktion für beide Spielfiguren berechnet, indem eine Rückwärts-BFS vom Ziel aus durchgeführt wird. Diese Distanzen dienen als Heuristik. Die Zustände werden in einer Prioritätswarteschlange (`heapq`) gespeichert und mit der Bewertungsfunktion $f = g + h$ priorisiert, wobei g die Distanz vom Start

Beim greedy-Ansatz werden nach dem Einlesen der Spielfeldgrößen, Bewegungsmöglichkeiten und Grubenpositionen für jede Figur die Felder mit einer Rückwärtssuche (BFS) vom Ziel aus bewertet. Dabei wird jedem Feld ein 4-Tupel zugeordnet, das die Qualität von Bewegungen in die vier Richtungen beschreibt. Die Rückwärtssuche erfolgt mit einer `deque`, wobei erreichbare Felder schrittweise markiert werden. Bewegungen gegen Wände werden mit 0, Wege, die zum Ziel hin führen mit 1 und Wege, die vom Ziel weg führen mit -1 charakterisiert. Dann wird vom Startpunkt aus iterativ die besteBewegungsanweisung gewählt, indem für jede mögliche Richtung die Summe der Auswirkungen auf alle Labyrinthe berechnet wird. Die Richtung mit maximalem positivem Gesamteinfluss wird gewählt. Um Zyklen zu vermeiden werden schon besuchte Zustände in einer Menge `seen` gespeichert und dürfen nicht erneut besucht werden. Überlegen wir uns nun, wie wir mit Gruben in diesem Fall umgehen. Gruben führen oft vom Ziel weg und sollten daher nur falls unbedingt nötig genommen werden. Dieser Fall tritt genau dann ein, wenn wir nur noch zurück oder in eine Grube gehen können. Was ist der schlechteste Score, den ein Weg ohne Gruben erreichen kann? Die Antwort ist offensichtlich -2, wenn beide Pfade vom Ziel weg führen. Ein Weg, der in eine Grube führt, sollte schlechter bewertet sein, selbst wenn wir dadurch im anderen Labyrinth vorankommen, da wir uns lieber in beiden Labyrinthen ein bisschen vom Ziel entfernen, als in einem der beiden extrem weit. Daher werden Gruben mit -4 bewertet. Kann kein neuer gültiger Zustand gefunden werden, wird der Suchvorgang abgebrochen. Am Ende werden die Bewegungsfolge, deren Länge und die Berechnungszeit ausgegeben.

4 Erweiterung

Die Problemstellung lässt sich von 2 auch auf l Labyrinthe erweitern. Bei den BFS- und A^*-Algorithmen hätten wir dann einen Zustandsraum mit $O(N^l)$ Elementen, was zu einer Laufzeit von $O(N^l)$ bzw. $O(N^l + \log N^l) = O(N^l + l \log N)$. Obwohl dies für konstantes l zwar noch polynomiell ist, steigt diese Laufzeit relativ schnell und ist nicht praktikabel.
Beim Greedy-Ansatz hingegen bleibt die Laufzeit $O(lN)$, da die Anzahl der Entscheidungen linear zur Weglänge bleibt und nur die Bewertung der möglichen Züge aufwändiger wird. Konkret müsste für jede der vier Bewegungsrichtungen die Summe der Veränderungen in allen l Labyrinthen berechnet werden, was eine konstante Laufzeitvergrößerung um einen Faktor l bedeutet. Damit bleibt der Greedy-Algorithmus für große l praktikabel, auch wenn seine Lösungsqualität tendenziell schlechter wird, da lokale Optimierung über viele Labyrinthe hinweg häufiger zu global ungünstigen Entscheidungen führen kann. Der A*- sowie der greedy- Algorithmus wurden hierfür erweitert. Da in den normalen Inputdateien kein Indikator für die Anzahl der Labyrinthe ist, muss diese Anzahl gehardcoded werden. Dafür änderen SIe einfach die Variable `num_mazes` von 2 auf Ihren Wunschwert.

5 Beispiele

Aus Gründen der besseren Lesbarkeit werden nur die Distanzen zum Ziel angegeben. Die Weglänge ??? steht dafür, dass mein Programm hier keine Lösung finden konnte. Die vollständigen Wege entnehmen Sie bitte den angehängten Text-Dateien. In diesen befindet sich die Weglänge und danach der Weg in Form einer Liste. Dabei bedeutet 0 nach rechts, 1 nach unten, 2 nach links und 3 nach oben. Die Datei labyrinthe01 enthält drei Labyrinthe. Bei labyrinthe4 konnten BFS und A* nur dann eine Lösung liefern, wenn man den Pfad nicht mit berechnete (aufgrund von erhöhtem Speicherverbrauch).

Weglängen:

Eingabe	A*	BFS	greedy
labyrinthe0:	8	8	10
labyrinthe1:	31	31	38
labyrinthe2:	65	65	93
labyrinthe3:	164	164	228
labyrinthe4:	14384	14384	18960
labyrinthe5:	???	???	???
labyrinthe6:	1878	???	???
labyrinthe7:	k.Lsg.	k.Lsg.	k.Lsg.
labyrinthe8:	???	???	786
labyrinthe9:	???	???	1398
labyrinthe01:	10	-	18

6 Quellcode

BFS:

```
def compress_state(state):
    (pos1,pos2)=state
    i1, j1 = pos1
    i2, j2 = pos2
    index1 = i1 * n + j1
    index2 = i2 * n + j2
    return index1 * total_cells + index2

n,m=(int(i) for i in input().split())
total_cells = m * n

right=[]
down=[]
gruben=[set(),set()]

for k in (0,1):
    right.append([[int(j) for j in input().split()] for i in range(m)])
    down.append([[int(j) for j in input().split()] for i in range(m-1)])
```

```
    for i in range(int(input())):
        x=[int(j) for j in input().split()]
        gruben[k].add((x[1],x[0]))

t0=time.time()
reach=[]

for k in (0,1):
    reach.append([])
    for i in range(m):
        line=[]
        for j in range(n):
            results=[]
            if j+1<n and not right[k][i][j]:
                if (i,j+1) in gruben[k]:
                    results.append((0,0))
                else:
                    results.append((i,j+1))
            else: results.append((i,j))
            if i+1<m and not down[k][i][j]:
                if (i+1,j) in gruben[k]:
                    results.append((0,0))
                else:
                    results.append((i+1,j))
            else: results.append((i,j))
            if 1<=j and not right[k][i][j-1]:
                if (i,j-1) in gruben[k]:
                    results.append((0,0))
                else:
                    results.append((i,j-1))
            else: results.append((i,j))
            if 1<=i and not down[k][i-1][j]:
                if (i-1,j) in gruben[k]:
                    results.append((0,0))
                else:
                    results.append((i-1,j))
            else: results.append((i,j))
            line.append(results)
        reach[k].append(line)


seen={0}
queue = deque([(((0,0), (0,0)),[])])

dist=0

while ((m-1,n-1),(m-1,n-1)) not in queue:
    for _ in range(len(queue)):
        node, path = queue.popleft()
        p0,p1=node
        x=reach[0][p0[0]][p0[1]] if p0!=(m-1,n-1) else [((m-1),(n-1)),((m-1),(n-1)),((m-1),(n-1)),((m-1
        y=reach[1][p1[0]][p1[1]] if p1!=(m-1,n-1) else [((m-1),(n-1)),((m-1),(n-1)),((m-1),(n-1)),((m-1
        for i in range(4):
            next_state = (x[i], y[i])
            compressed_state = compress_state(next_state)
            if compressed_state not in seen:
                seen.add(compressed_state)
                queue.append((next_state, path+[i]))
                if next_state==((m-1,n-1),(m-1,n-1)):
                    print(dist+1)
                    print(path)
                    quit()
    dist+=1
    if not queue:
        print('NO SOLUTION FOUND')
        break
```

A-Stern:

```
n,m=(int(i) for i in input().split())
total_cells = m * n
```

```python

right=[]
down=[]
gruben=[]

for l in range(num_mazes):
    right.append([[int(j) for j in input().split()] for i in range(m)])
    down.append([[int(j) for j in input().split()] for i in range(m-1)])
    gruben.append(set())

    for i in range(int(input())):
        x=[int(j) for j in input().split()]
        gruben[l].add((x[1],x[0]))

t0=time.time()
reach=[]
goal=(m-1,n-1)

for l in range(num_mazes):
    reach.append(dict())
    for i in range(m):
        for j in range(n):
            results=[]
            if j+1<n and not right[l][i][j]:
                if (i,j+1) in gruben[l]:
                    results.append((0,0))
                else:
                    results.append((i,j+1))
            else: results.append((i,j))
            if i+1<m and not down[l][i][j]:
                if (i+1,j) in gruben[l]:
                    results.append((0,0))
                else:
                    results.append((i+1,j))
            else: results.append((i,j))
            if 1<=j and not right[l][i][j-1]:
                if (i,j-1) in gruben[l]:
                    results.append((0,0))
                else:
                    results.append((i,j-1))
            else: results.append((i,j))
            if 1<=i and not down[l][i-1][j]:
                if (i-1,j) in gruben[l]:
                    results.append((0,0))
                else:
                    results.append((i-1,j))
            else: results.append((i,j))
            reach[l][(i,j)]=results
    reach[l][goal]=[goal,goal,goal,goal]

dist=[] #contains the distance from the goal for each position in the grid
goal=(m-1,n-1)

for l in range(num_mazes):
    cright=right[l]
    cdown=down[l]
    cd=dict()

    queue=deque([goal])
    seen={goal}

    d=0

    while queue:
        for _ in range(len(queue)):
            pos=queue.popleft()
            a=[]
            y,x=pos

            if not (y==m-1 or cdown[y][x] or (y+1,x) in seen):
                queue.append((y+1,x))
                seen.add((y+1,x))

```

```
                if not (x==n-1 or cright[y][x] or (y,x+1) in seen):
                    queue.append((y,x+1))
                    seen.add((y,x+1))

                if not(y==0 or cdown[y-1][x] or (y-1,x) in seen):
                    queue.append((y-1,x))
                    seen.add((y-1,x))

                if not(x==0 or cright[y][x-1] or (y,x-1) in seen):
                    queue.append((y,x-1))
                    seen.add((y,x-1))

                cd[pos]=d
            d+=1
        dist.append(cd)


start=num_mazes*((0,0),)

open_list=[]
seen={start}

heapq.heappush(open_list,(max(dist[j][start[j]] for j in range(num_mazes)),0,start,[]))
dmin=float('inf')

while open_list:
    s,g,pos, path = heapq.heappop(open_list)
    if s==g:
        break
    g+=1
    for i in range(4):
        npos=[]
        for l in range(num_mazes):
            npos.append(reach[l][pos[l]][i])
        npos=tuple(npos)
        if not npos in seen:
            f=max(dist[j][npos[j]] for j in range(num_mazes))
            heapq.heappush(open_list,(f+g,g,npos,path+[i]))
            seen.add(npos)

if s==g:
    print(g)
    print(path)
else:
    print('NO SOLUTION FOUND')
```

Heuristik:

```
n, m = map(int, input().split())

right = []
down = []
gruben = []

for k in range(num_mazes):
    right.append([[int(j) for j in input().split()] for _ in range(m)])
    down.append([[int(j) for j in input().split()] for _ in range(m - 1)])
    gruben.append([])
    for _ in range(int(input())):
        gruben[k].append(tuple(map(int, input().split())))

t0=time.time()

goal = (m - 1, n - 1)

def compute_areas():
    areas = []
    dirs = [(1, 0), (0, 1), (-1, 0), (0, -1)]  # R, D, L, U in (dx, dy)
    for k in range(num_mazes):
        cright = right[k]
        cdown = down[k]
        traps = set(gruben[k])
```

```
        area = {}
        queue = deque([goal])
        seen = {goal}
        while queue:
            y, x = queue.popleft()
            vals = []
            for dx, dy in dirs:
                nx, ny = x + dx, y + dy
                # 1) Fallen
                if (nx, ny) in traps:
                    vals.append(-2 * num_mazes)
                # 2) Rand
                elif not (0 <= nx < n and 0 <= ny < m):
                    vals.append(0)
                # 3) Wand
                else:
                    if dx == 1 and cright[y][x] == 1:
                        vals.append(0)
                    elif dx == -1 and cright[y][x - 1] == 1:
                        vals.append(0)
                    elif dy == 1 and cdown[y][x] == 1:
                        vals.append(0)
                    elif dy == -1 and cdown[y - 1][x] == 1:
                        vals.append(0)
                    else:
                        if (ny, nx) in seen:
                            vals.append(1)
                        else:
                            vals.append(-1)
                            seen.add((ny, nx))
                            queue.append((ny, nx))
            area[(y, x)] = vals
        area[goal] = [0, 0, 0, 0]
        areas.append(area)
    return areas

# BFS-Berechnung
areas = compute_areas()

# Absicherung: Start erreichbar?
start = (0, 0)
for k in range(num_mazes):
    if start not in areas[k]:
        print(f"Labyrinth {k} vom Start aus nicht erreichbar!")
        exit()

# Pfadsuche
pos = tuple([start] * num_mazes)
final = tuple([goal] * num_mazes)
seen_states = set()
path = []
steps = 0
moves = [(1, 0), (0, 1), (-1, 0), (0, -1)]

while pos != final:
    seen_states.add(pos)
    steps += 1
    best_move = None
    best_score = -float('inf')
    for i, move in enumerate(moves):
        new = []
        score = 0
        valid = True
        for k in range(num_mazes):
            cell_scores = areas[k].get(pos[k], [ -2*num_mazes ] * 4)
            r = cell_scores[i]
            score += r
            if r == -2 * num_mazes:
                valid = False
            if r != 0:
                # Bewegung moeglich
                new.append((pos[k][0] + move[1], pos[k][1] + move[0]))
            else:
```

```
                new.append(pos[k])
        new_pos = tuple(new)
        if valid and new_pos not in seen_states and score > best_score:
            best_score = score
            best_move = (new_pos, i)

    if best_move is None:
        print('NO SOLUTION FOUND')
        exit()
    pos, mv = best_move
    path.append(mv)

print(len(path))
print(path)
```